Notes:

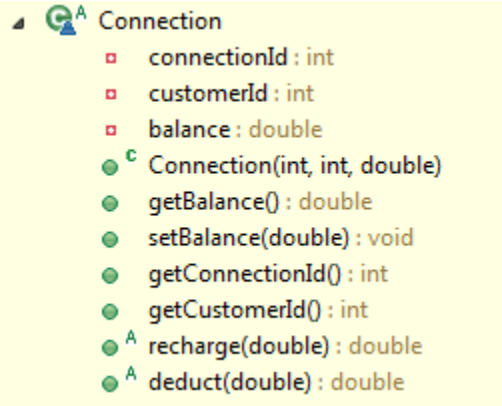
- Go through the problem statement clearly.
- Time limit is 150 minutes.
- Make sure that project is created in eclipse only.
- Create all your java files in package **com** within src folder of eclipse project.
- Make sure that exact class outline is followed as you did in previous assignments.
- You need to zip the eclipse project folder and upload the same in LMS once completed. The project folder will be available in your workspace folder.
- It is mandatory to upload eclipse project and not only java files for your code to be assessed.
- Make sure that there is no compilation error in your code before submission. Even if there is minor error, entire solution could be rejected.
- You may refer previous assignments, course content and internet for any reference.
- Last section is optional.

A telecom inventory system is looking for automated solution for managing connections and related recharge and balance deduction. The system would currently support feature of prepaid connections, however, it may have more connection types in future. Develop a Java project which serves this system following below guidelines.

Class outline is shared after the guidelines. Ensure the same is met 100%, else your solution would be considered null and void.

Create project TelecomSystem in eclipse. Create package **com** within src folder. All classes should be created inside this package.

Create abstract class – Connection with attributes connection id, customer id and **abstract methods** – recharge and deduct. Refer below class outline for this class:



Create child class – PrePaid from this abstract class. This class will implement recharge and deduct methods where amount to be recharged should be added to the balance and amount to be deducted should be deducted from the balance. Withdrawal will not check for zero balance, hence balance can go in negative range. Refer below outline for this class.

```
PrePaid
  PrePaid(int, int, double)
  recharge(double) : double
  deduct(double) : double
```

Create class Inventory which has ArrayList of Connection class. Initialize this list in the constructor.

Write below methods in Inventory class:

addConnection – This method will take Connection object as a parameter. It will add this object to the list if connection with specific id does not exist and return true. If connection with specific id already exists, it will return false.

recharge – This method will take connection id and recharge amount. Amount will be deposited into specific connection with that id. Method will return new balance if connection with that id exists, else returns -1.

deduct – This method will take connection id and deduct amount. It will return -1 if connection with specific id does not exist. Else, it will check if existing balance is not negative. If no, perform deduction and return new balance (which may be negative). However, existing balance is already negative, then exception of class "ConnectionLockedException" should be thrown. This class is explained later.

getUniqueCustomerIds() – This method will return a Set of Integer types, where it will contain unique customer ids across all Connection objects.

getConnectionHashMap – This method will return HashMap with connection id as key and respective connection object as value from existing connection objects in connection array list.

Refer below outline for this class.

```
Inventory
  connections : ArrayList<Connection>
  Inventory()
  addConnection(Connection) : boolean
  recharge(int, double) : double
  deduct(int, double) : double
  getUniqueCustomerIds() : Set<Integer>
  getConnectionHashMap() : HashMap<Integer, Connection>
```

Declare class ConnectionLockedException which extends Exception class.

Constructor of this class will take existing balance as parameter. getMessage() method should be overridden such that it that balance as a String. Outline of this class is as below:

```
ConnectionLockedException
    currentBalance : double
    ConnectionLockedException(double)
    getMessage() : String
```

Optional:

Declare class NewInventory which has tree set of Connection objects. Write method to add connection object to this set and returns size of the set after adding the object. Tree set of connection should maintain connections in ascending order of connection id.(Hint you would need to implement comparable in Connection class)

Refer class outline as below:

```
NewInventory
    connections : TreeSet<Connection>
    NewInventory()
    getConnections() : TreeSet<Connection>
    addConnection(Connection) : int
```

Declare a class ConnectionDemo which has main method. Test execution as below:

- Declare Inventory object.
- Declare three Connection objects with separate connection ids(PrePaid connection objects referred by Connection references)
- Add these objects in Inventory object.
- Test recharge method of Inventory object
- Test deduct method of bank object along with the case where ConnectionLockedException is thrown.
- Test remaining methods of Inventory object.
- Create fourth connection object with same connection id matching connection objects created before.
- Try adding this connection to Inventory object.
- If optional part is implemented, then create object of NewInventory class object and test execution of add connection method by adding connection objects in different order of connection ids.